If you have an HTML string and do not want Razor to encode it, you can use the `Html.Raw` method to prevent encoding:

```
@Html.Raw(“<p>This is a paragraph.</p>”)
```

This is useful for scenarios in which, for example, you are storing HTML in a database for display—a typical requirement for most content management systems and the like. However, you should ensure that the HTML is sanitized prior to including it in your page. Otherwise, you could open your site up to script injection attacks. You will learn about these and other vulnerabilities in chapter 13.

The aggressive level of encoding that is applied by default may not be desirable for developers of sites that store and output content in languages that make heavy or exclusive use of non-ASCII characters. Take this snippet of German as an example (it translates to “Charges for oversized luggage”):

```
var output = "Gebühren für übergroßes Gepäck";
```

This is encoded to the following when it is embedded in the markup as `@output`:

```
Geb&#xFC;hren f&#xFC;r &#xFC;bergro&#xDF;es Gep&#xE4;ck
```

For non-Latin based languages, such as Cyrillic, Chinese, and Arabic, every character is encoded to its HTML equivalent, potentially increasing the number of characters in the generated source code significantly. While the output renders correctly, the page’s resulting size and overall site performance can be affected adversely.

It is possible to arrange for a broader range of characters, so they are not encoded. You can do this in Program.cs by setting the `WebEncoderOptions`. By default, the allowed range of characters (i.e., those not encoded) is restricted to the Basic Latin character set. Listing 3.10 demonstrates how to configure the options